Layer 4 – Technical Specification Document

Document Version: 1.0
Date: November 10, 2025
Author: Zbigniew Szymon Kołacz, Stoic MatriX
Purpose: Complete technical blueprint for MVP development

1. System Architecture Overview

High-Level Flow

```
User Input (Reflection)
    ↓
FastAPI Backend
    ├→ NLP Pipeline (Language detection)
    ├→ Transformers (Sentiment + Topic classification)
    ├→ Nudge Engine (Quote + Exercise selection)
    ├→ AI Coach (GPT-4 context + response)
    └→ Database Layer (PostgreSQL + MongoDB)
    ↓
React Frontend
    ├→ Dashboard (Results display)
    ├→ Trends (Historical visualization)
    ├→ Sympozion (Community features)
    ├→ Gamification (Badges, streaks)
    └→ AI Coach Chat (WebSocket streaming)
    ↓
External Integrations
    ├→ Obsidian Plugin
    ├→ Notion API
    ├→ Google Calendar API
    └→ Email/Push Notifications
```

2. Backend Specification (FastAPI)

2.1 Project Structure

```
backend/
├── app/
│   ├── __init__.py
│   ├── main.py                    # FastAPI app initialization
│   ├── config.py                  # Configuration, environment variables
│   ├── database.py                # SQLAlchemy setup
│   │
│   ├── models.py                  # Database models
│   │   ├── User
│   │   ├── UserReflection
│   │   ├── UserAchievement
│   │   ├── UserStreak
│   │   ├── CoachConversation
│   │   ├── StudyGroup
│   │   ├── GroupDiscussion
│   │   ├── GroupChallenge
│   │   └── ChallengePerson
│   │
│   ├── schemas.py                 # Pydantic models (request/response)
│   │   ├── AnalyzeRequest
│   │   ├── AnalyzeResponse
│   │   ├── UserCreate
│   │   └── ...
│   │
│   ├── sentiment.py               # NLP/ML logic
│   │   ├── detect_language()
│   │   ├── analyze_text()
│   │   ├── classify_topic()
│   │   └── score_keywords()
│   │
│   ├── nudge.py                   # Nudge/coaching generation
```

```
│   │   ├── generate_nudge()
│   │   └── NUDGE_DB, EXERCISE_DB
│   │
│   ├── badges.py                 # Gamification engine
│   │   ├── GamificationEngine
│   │   └── award_badge()
│   │
│   ├── ai_coach.py               # AI coaching logic
│   │   ├── StoicAICoach
│   │   └── chat(), get_exercise()
│   │
│   ├── sympozion.py              # Community features
│   │   ├── SympozionEngine
│   │   └── create_group(), join_challenge()
│   │
│   ├── persistence.py            # Database operations
│   │   └── save_reflection()
│   │
│   ├── auth.py                   # JWT authentication
│   │   └── get_current_user()
│   │
│   └── routers/
│       ├── analysis.py           # /analyze endpoints
│       ├── history.py            # /history endpoints
│       ├── coach.py              # /coach endpoints
│       ├── achievements.py       # /achievements endpoints
│       └── sympozion.py          # /sympozion endpoints
│
├── tests/
│   ├── test_sentiment.py
│   ├── test_nudge.py
│   ├── test_badges.py
│   └── test_api.py
│
├── requirements.txt
├── Dockerfile
├── docker-compose.yml
└── .env.example
```

2.2 Key Endpoints

Analysis

```
POST /analyze
  Request: { "text": "...", "language": "auto" }
  Response: {
    "sentiment": 0.82,
    "language": "pl",
    "topic": "wdzięczność",
    "quote": "...",
    "exercise": "...",
    "achievements": { "badges": [...], "total_points": 50 }
  }

GET /history?user_id=...&amp;limit=20
  Response: [{ reflection_id, timestamp, sentiment, topic, ... }, ...]

GET /streaks
  Response: [{ type, current, longest, last_activity }, ...]
```

AI Coach

```
WebSocket /ws/coach/{user_id}
  Streaming messages to/from AI Coach
  Real-time responses with context

GET /coach/exercise?focus_area=gratitude
  Response: { "name", "purpose", "steps", "reflection_questions", ... }

POST /coach/reflect
  Request: { "reflection_text": "...", "analysis_result": {...} }
  Response: { "coaching": "..." }
```

Sympozion

```
POST /sympozion/group
  Request: { "name", "description", "topic", "is_public": true }
  Response: { "group_id", "name" }

GET /sympozion/groups
  Response: [{ "id", "name", "members_count", "focus_topic" }, ...]

POST /sympozion/group/{group_id}/challenge
  Request: { "title", "description", "topic", "duration_days": 7 }
  Response: { "challenge_id" }

GET /sympozion/challenge/{challenge_id}/leaderboard
  Response: { "leaderboard": [{ "user_id", "score", "completed" }, ...] }
```

2.3 Database Schema

user_reflections

```
id (PK)
user_id (FK)
timestamp
text (VARCHAR(5000))
sentiment (FLOAT 0-1)
language (VARCHAR(8))
topic (VARCHAR(32))
quote (TEXT)
exercise (TEXT)
```

user_achievements

```
id (PK)
user_id (FK)
badge_id (VARCHAR(64))
earned_at (TIMESTAMP)
progress (INT 0-100)
```

study_groups

```
id (PK)
name (VARCHAR(255))
description (TEXT)
founder_id (FK)
created_at
is_public (BOOLEAN)
max_members (INT)
focus_topic (VARCHAR(32))
```

3. Frontend Specification (React 18 + TypeScript)

3.1 Component Structure

```
src/
├── components/
│   ├── Auth/
│   │   ├── LoginForm.tsx
│   │   └── RegisterForm.tsx
│   │
│   ├── Analysis/
│   │   ├── AnalyzeForm.tsx       # Textarea + submit
│   │   ├── ResultDisplay.tsx      # Show score, quote, exercise
│   │   └── HistoryList.tsx        # Previous reflections
│   │
│   ├── Dashboard/
│   │   ├── Dashboard.tsx          # Main page layout
│   │   └── NavBar.tsx
│   │
```

```
│   ├── Gamification/
│   │   ├── BadgesDisplay.tsx      # Badge grid
│   │   ├── StreakDisplay.tsx      # Streak cards
│   │   └── AchievementNotification.tsx
│   │
│   ├── AICoach/
│   │   ├── ChatWindow.tsx         # WebSocket chat
│   │   ├── ExerciseGenerator.tsx  # Call /coach/exercise
│   │   └── CoachingPanel.tsx
│   │
│   ├── Trends/
│   │   ├── TrendsChart.tsx        # Line chart (Recharts)
│   │   ├── TopicStats.tsx         # Pie chart
│   │   └── WeeklySummary.tsx      # Summary + plan
│   │
│   ├── Onboarding/
│   │   └── OnboardingWizard.tsx   # 7-step wizard
│   │
│   └── Sympozion/
│       ├── SympozionHub.tsx
│       ├── GroupList.tsx
│       ├── GroupDetail.tsx
│       ├── ChallengeBoard.tsx
│       └── Leaderboard.tsx
│
├── services/
│   ├── api.ts                     # Axios client, all API calls
│   └── websocket.ts               # WebSocket management
│
├── hooks/
│   ├── useAuth.ts
│   ├── useUser.ts
│   └── useWebSocket.ts
│
├── styles/
│   ├── global.css
│   ├── components.css
│   └── theme.css
│
└── App.tsx                        # Main router
```

3.2 Key Features

AnalyzeForm Component

- Rich textarea for reflection input
- Auto-detect language option
- Loading state while API processes
- Error handling with user feedback

TrendsChart Component

- Line chart showing virtue score over 30 days
- Highlight trends (↑ positive, ↓ negative)
- Tooltip on hover showing exact date + score
- Export to CSV button

ChatWindow Component

- Message history (user messages on right, coach on left)
- WebSocket streaming (real-time responses)
- Typing indicator
- Input field with send button
- Auto-scroll to newest message

4. NLP/ML Pipeline

4.1 Language Detection

```
from langdetect import detect

def detect_language(text: str) -&gt; str:
    try:
        lang = detect(text)
        return 'pl' if lang == 'pl' else 'en'
    except:
        return 'en'  # Fallback
```

4.2 Sentiment Analysis

```
from transformers import pipeline

# EN model
sentiment_en = pipeline(
    "sentiment-analysis",
    model="distilbert-base-uncased-finetuned-sst-2-english"
)

# PL model
sentiment_pl = pipeline(
    "sentiment-analysis",
    model="allegro/herbert-base-cased-sentiment"
)

def analyze_sentiment(text: str, lang: str) -&gt; float:
    model = sentiment_pl if lang == "pl" else sentiment_en
    result = model(text)[0]
    return float(result['score'])
```

4.3 Topic Classification (Zero-shot)

```
from transformers import pipeline

zero_shot = pipeline(
    "zero-shot-classification",
    model="facebook/bart-large-mnli"
)

TOPICS_PL = ["wdzięczność", "odwaga", "cierpliwość", "akceptacja", "siła"]
TOPICS_EN = ["gratitude", "courage", "patience", "acceptance", "strength"]

def classify_topic(text: str, lang: str) -&gt; str:
    labels = TOPICS_PL if lang == "pl" else TOPICS_EN
    result = zero_shot(text, labels)
    return result['labels'][0]  # Top label
```

4.4 Custom Keyword Fallback

```
PL_POS = ["wdzięczność", "cnota", "siła", "dzielność", "mądrość"]
PL_NEG = ["lęk", "złość", "smutek", "niesprawiedliwość"]

def score_keywords(text: str, pos: list, neg: list) -&gt; float:
    plus = sum(1 for word in pos if word in text.lower())
    minus = sum(1 for word in neg if word in text.lower())
    return min(max((plus - minus + 2) / 4, 0), 1)
```

5. AI Coach Integration

5.1 LangChain Setup

```
from langchain.chat_models import ChatOpenAI
from langchain.memory import ConversationBufferWindowMemory
from langchain.chains import ConversationChain

class StoicAICoach:
    def __init__(self, user_id: str):
        self.llm = ChatOpenAI(
            model_name="gpt-4",
            temperature=0.7,
            openai_api_key=os.getenv("OPENAI_API_KEY")
        )
        self.memory = ConversationBufferWindowMemory(k=10)
        self.conversation = ConversationChain(
            llm=self.llm,
            memory=self.memory
        )

    def chat(self, user_message: str) -&gt; str:
        response = self.conversation.predict(input=user_message)
        return response
```

5.2 WebSocket Streaming

```
@app.websocket("/ws/coach/{user_id}")
async def websocket_coach(websocket: WebSocket, user_id: str):
    await websocket.accept()
    coach = StoicAICoach(user_id)

    while True:
        data = await websocket.receive_text()
        # Stream response in chunks
        full_response = coach.chat(data)
        await websocket.send_text(full_response)
```

6. Gamification System

6.1 Badge Definitions

```
BADGES = {
    "first_analysis": {
        "name": "First Step",
        "description": "Completed your first analysis",
        "icon": "",
        "points": 10
    },
    "7_day_streak": {
        "name": "Week Warrior",
        "description": "7 consecutive days",
        "icon": "",
        "points": 50
    },
    # ... more badges
}
```

6.2 Streak Logic

```
def update_streak(user_id: str, streak_type: str):
    streak = db.query(UserStreak).filter_by(
        user_id=user_id,
        streak_type=streak_type
    ).first()

    today = datetime.utcnow().date()
    last_activity = streak.last_activity.date() if streak else None

    if last_activity == today - timedelta(days=1):
```

```
        streak.current_streak += 1  # Continue
    else:
        streak.current_streak = 1   # Reset

    streak.longest_streak = max(streak.longest_streak, streak.current_streak)
    db.commit()
```

7. Testing Strategy

7.1 Unit Tests (PyTest)

```
def test_detect_language():
    assert detect_language("Dziękuję") == "pl"
    assert detect_language("Thank you") == "en"

def test_analyze_sentiment():
    score = analyze_sentiment("I am grateful", "en")
    assert 0.7 &lt;= score &lt;= 1.0

def test_classify_topic():
    topic = classify_topic("I appreciate this moment", "en")
    assert topic == "gratitude"
```

7.2 E2E Tests (Cypress)

```
describe("Full analysis flow", () =&gt; {
    it("should analyze reflection and display results", () =&gt; {
        cy.visit("/dashboard");
        cy.get("textarea").type("I am grateful today");
        cy.contains("Analyze").click();
        cy.contains("Virtue Score").should("be.visible");
    });
});
```

7.3 Load Testing

- Locust for API stress testing
- Test 1000 concurrent users
- Monitor response times, error rates

8. Deployment & Infrastructure

8.1 Docker Setup

Dockerfile (Backend)

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY ./app ./app
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0"]
```

docker-compose.yml

```
version: "3.9"
services:
  backend:
    build: .
    ports:
      - "8000:8000"
    depends_on:
      - db
    environment:
      - DATABASE_URL=postgresql://postgres:pass@db/layer4
```

```
  db:
    image: postgres:15
    environment:
      POSTGRES_DB: layer4
      POSTGRES_PASSWORD: pass
    volumes:
      - db_data:/var/lib/postgresql/data

volumes:
  db_data:
```

8.2 CI/CD Pipeline (GitHub Actions)

```
name: CI/CD
on: [push]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Run backend tests
        run: cd backend &amp;&amp; pytest
      - name: Run frontend tests
        run: cd frontend &amp;&amp; npm test
      - name: Build Docker image
        run: docker build -t layer4:latest .
      - name: Push to registry
        run: docker push yourdockerhub/layer4:latest
```

9. Performance & Optimization

9.1 Model Optimization

- Use DistilBERT (4x faster than BERT, 40% smaller)
- Cache models in memory (avoid reload)
- Batch processing for high volume

9.2 Database Optimization

- Index on `user_id`, `timestamp`
- Connection pooling (SQLAlchemy pool_size=20)
- Query optimization for trends (materialized views)

9.3 Frontend Optimization

- Code splitting (React.lazy for components)
- Memoization (useMemo, useCallback)
- Image optimization (WebP, lazy loading)

10. Security

10.1 Authentication

- JWT tokens (refresh + access)
- OAuth2 support (Google, GitHub)
- Rate limiting (FastAPI limits)

10.2 Data Protection

- End-to-end encryption for sensitive data
- HTTPS everywhere
- GDPR compliance (right to deletion, data export)

10.3 API Security

- Input validation (Pydantic)
- SQL injection prevention (ORM)
- CORS properly configured

11. Monitoring & Logging

11.1 Error Tracking (Sentry)

```
import sentry_sdk
sentry_sdk.init(dsn=SENTRY_DSN)
```

11.2 Metrics (Prometheus)

```
from prometheus_client import Counter, Histogram

analysis_counter = Counter('analyses_total', 'Total analyses')
response_time = Histogram('response_time_seconds', 'Response time')
```

11.3 Logs

- JSON structured logging
- Log levels: DEBUG, INFO, WARNING, ERROR
- Centralized via ELK stack (Elasticsearch, Logstash, Kibana)

12. Development Workflow

12.1 Environment Setup

```
git clone layer4-api
cd layer4-api
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
python -m pytest
```

12.2 Branch Strategy

- `main` – production
- `develop` – staging
- `feature/*` – feature branches
- PR review required before merge

12.3 Release Process

1. Bump version (semantic versioning)
2. Merge to main, tag release
3. GitHub Actions builds & pushes Docker image
4. Deploy to production (rolling update)

13. API Rate Limiting & Quotas

Tier	Requests/hour	Max analyses/day
Free	60	3
Premium	1000	unlimited
Enterprise	10,000+	unlimited

14. Third-party Integrations

14.1 OpenAI API

- GPT-4 for AI Coach
- Cost: ~$0.03 per 1K tokens
- Caching for cost efficiency

14.2 Obsidian Plugin

- Uses Layer 4 API
- OAuth2 for auth
- Local caching for offline support

14.3 Notion API

- Sync reflections to Notion databases
- Read/write permissions required
- Rate limited to 3 requests/second

15. Appendix: Key Dependencies

```
# Backend
fastapi==0.104.1
uvicorn==0.24.0
sqlalchemy==2.0.23
transformers==4.34.0
langchain==0.1.0
pydantic==2.5.0
psycopg2-binary==2.9.9
python-dotenv==1.0.0
pyjwt==2.8.1

# Frontend
react==18.2.0
typescript==5.3.3
axios==1.6.2
recharts==2.10.1
socket.io-client==4.7.2

# Testing
pytest==7.4.3
pytest-asyncio==0.21.1
cypress==13.6.2
jest==29.7.0
```

Document ends here. All technical specifications are ready for development.

Ready to build Layer 4. Let's transform consciousness.